

Chapter 2

Intelligent Agents

2.1 Overview of Intelligent Agents

This chapter introduces the concept of intelligent agents, which serve as the foundational unifying framework for modern artificial intelligence. It examines how agents perceive their surroundings through sensors, process information, and execute actions through actuators within diverse operational environments. The chapter explores the criteria for agent rationality, formalizes task environment specifications using the Performance, Environment, Actuators, and Sensors (PEAS) framework, and analyzes fundamental agent architectures ranging from simple reflex systems to sophisticated learning agents.

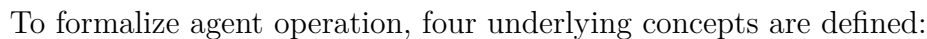
Understanding intelligent agents is critical because it shifts the focus of artificial intelligence from isolated algorithms to holistic, goal-oriented decision-making systems. Rather than viewing intelligence merely as a collection of specialized techniques, such as search or logic, the agent-centric viewpoint provides a principled structure for designing autonomous systems that operate effectively in complex, dynamic, and uncertain real-world domains. Studying agent architectures allows engineers and computer scientists to evaluate trade-offs between computational overhead, reactivity, adaptability, and decision accuracy.

Building upon the historical and theoretical foundations established in Chapter 1, this chapter establishes the design paradigm used throughout the remainder of CSC 309. The environment classifications and structural components introduced here provide the context for problem-solving by search algorithms in Chapter 3, formal Knowledge representation models in Chapter 4, and specialized domain applications such as Natural Language Processing, expert systems, and computer vision in subsequent chapters.

2.1.1 Definition and Core Concepts

An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators. A human agent possesses physical sensors such as eyes, ears, and nose, alongside actuators such as hands, legs, and vocal cords. A robotic agent replaces biological sensors with cameras, infrared rangefinders, and lidar units, and replaces biological actuators with electric motors, hydraulic limbs, and grippers.

An **intelligent agent** is an agent that selects actions expected to maximize its performance measure based on its percept sequence and built-in knowledge.



- ### 2.1.2 Agent Functions and Agent Programs

2

of all possible actions. The agent function f is defined as:

$$f : \mathcal{P}^* \rightarrow \mathcal{A} \quad (2.1)$$

The agent function is an abstract mathematical mapping. In practice, the agent function is implemented by an agent program, which runs on a physical computing infrastructure known as the **agent architecture**.

While the agent function takes the entire history of percepts $\mathcal{P}^*$ as input, the agent program accepts only the current percept $p \in \mathcal{P}$ as an input from the sensors because storing and searching an unbounded percept sequence directly is computationally infeasible. The agent program maintains internal memory structures to represent relevant aspects of past percepts when necessary.

To illustrate this distinction, consider a simplified automated vacuum cleaner domain containing two locations, designated as Room A and Room B. The agent perceives its current location and whether that location contains dirt. The percept is represented as a pair, such as (Location A, Clean) or (Location B, Dirty). The available actions are Move Left, Move Right, Suck, and No-Op.

Percept Sequence	Action Choice
[(Location A, Clean)]	Move Right
[(Location A, Dirty)]	Suck
[(Location B, Clean)]	Move Left
[(Location B, Dirty)]	Suck
[(Location A, Clean), (Location B, Dirty)]	Suck
[(Location A, Clean), (Location B, Clean)]	No-Op

Table 2.1: Partial tabular representation of an agent function for a two-room vacuum world.

Table 2.1 demonstrates that the agent function grows exponentially with time if represented as a lookup table. For an agent operating over t time steps with $|\mathcal{P}|$ possible percepts, the number of entries in a complete lookup table is given by:

$$\sum_{k=1}^t |\mathcal{P}|^k = \frac{|\mathcal{P}|(|\mathcal{P}|^t - 1)}{|\mathcal{P}| - 1} \quad (2.2)$$

This exponential growth illustrates why explicit lookup tables are intractable for real-world artificial intelligence applications, necessitating structured agent programs.

2.2 Rationality and Agent Performance

Rationality evaluates whether an agent makes appropriate decisions given what it perceives and what it knows. Rationality is distinct from perfection.

2.2.1 Performance Measures

A performance measure is an objective criterion used to evaluate how successful an agent's behavior is within a given environment. It is crucial that performance measures are defined in terms of desired outcomes in the environment rather than in terms of the agent's internal state or actions.

For example, evaluating a vacuum cleaner agent based on the amount of dirt it collects might encourage an agent to repeatedly dump dirt and suck it back up to maximize its score. A sound performance measure evaluates the clean state of the environment over time, penalizing excessive energy consumption and noise.

2.2.2 Defining Rationality

What is rational at any given time depends on four distinct factors:

1. The performance measure that defines the criterion of success.
2. The agent's prior knowledge of the environment.
3. The actions that the agent can perform.
4. The agent's percept sequence to date.

Based on these factors, a rational agent is defined formally as follows: For each possible percept sequence, a rational agent selects an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent possesses.

Mathematically, if S represents the set of possible environment states, $U(s)$ denotes the utility or performance score of state $s \in S$, and $P(s \mid \mathcal{P}^*, a)$ represents the probability distribution over resulting states given percept sequence $\mathcal{P}^*$ and selected action $a \in \mathcal{A}$, the rational action choice a^* satisfies:

$$a^* = \arg \max_{a \in \mathcal{A}} \sum_{s \in S} P(s \mid \mathcal{P}^*, a) U(s) \quad (2.3)$$

2.2.3 Rationality versus Omniscience

Rationality is not synonymous with omniscience. An omniscient agent knows the actual outcome of its actions and acts accordingly. Omniscience is impossible in real-world applications because environments are uncertain and incomplete information is inherent to perceptual mechanisms.

Rationality maximizes **expected** performance, whereas omniscience maximizes **actual** performance. An agent decision that leads to an undesirable outcome due to unforeseen events is still rational if the chosen action was optimal based on the evidence available prior to execution.

Consider an individual walking down a designated pedestrian pathway who is struck by a falling piece of aircraft debris. The decision to walk along the path remains rational

because prior knowledge and sensory inputs provided no reasonable basis to predict the event. Expecting the agent to have avoided the location demands omniscience, not rationality.

2.2.4 Autonomy and Learning

An agent demonstrates autonomy to the extent that its choices depend on its own experience rather than on pre-programmed designer knowledge. An agent that relies entirely on built-in assumptions lacks flexibility and fails when deployed in changing environments.

A rational agent must possess two secondary mechanisms to support long-term operational success:

- **Information Gathering:** Taking actions specifically designed to expand the percept sequence and reduce uncertainty, such as looking around before crossing a street.
- **Learning:** Modifying initial knowledge structures based on environmental feedback over time, transforming a non-autonomous agent into an autonomous system.

2.3 Task Environments and the PEAS Framework

In agent design, the problem specification is termed the task environment. The framework used to specify a task environment is known as the PEAS framework.

2.3.1 The PEAS Framework

PEAS is an acronym representing four critical components:

- **Performance Measure:** The explicit metrics used to rate agent success.
- **Environment:** The external medium, domain, or context within which the agent operates.
- **Actuators:** The set of tools or output interfaces through which the agent acts.
- **Sensors:** The set of devices or input interfaces through which the agent receives percepts.

To design an effective agent program, the PEAS specification must first be clearly articulated.

2.3.2 Environment Properties

Task environments exhibit key structural dimensions that dictate the complexity of the required agent architecture:

Fully Observable versus Partially Observable

An environment is fully observable if an agent's sensors provide access to the complete state of the environment at each point in time. An environment is partially observable if sensors are noisy, inaccurate, or unable to detect aspects of the state due to physical limitations or

Agent Type	Performance	Environment	Actuators	Sensors
Automated Taxi Driver	Safety, speed, legal compliance, passenger comfort, profit	Urban roads, vehicular traffic, pedestrians, weather conditions	Steering, accelerator, brakes, signals, horn, display screen	Cameras, radar, lidar, GPS, accelerometer, engine sensors
Medical Diagnosis System	Correct diagnosis, minimized treatment costs, patient health	Patient body, clinical laboratory, healthcare staff	Displayed diagnosis, test recommendations, treatment plans	Keyboard entry of symptoms, laboratory test results
Satellite Image Classifier	Classification accuracy, throughput, low processing latency	Continuous satellite imagery stream	Categorization labels, anomaly warning flags	High-resolution optical sensors, infrared channels
Interactive English Tutor	Maximized student test performance, user engagement	Student cohort, curriculum database	Screen display of exercises, feedback text, speech output	Keyboard input, touchscreen interactions, microphone

Table 2.2: PEAS specifications across diverse application domains.

occlusions. In a partially observable domain, the agent must maintain an internal state to track unobserved elements.

Single-Agent versus Multi-Agent

An environment is single-agent if an entity acts alone without other decision-making agents affecting performance. An environment is multi-agent if other agents exist whose actions affect the performance measure of the primary agent. Multi-agent environments are categorized as competitive (such as chess) or cooperative (such as collaborative traffic management systems).

Deterministic versus Stochastic

An environment is deterministic if the next state of the environment is completely determined by the current state and the action executed by the agent. If uncertainty exists regarding the resulting state due to random elements or unobserved dynamics, the environment is stochastic. If the environment is not deterministic but outcomes are listed without explicitly attached probabilities, it is termed nondeterministic.

Episodic versus Sequential

In an episodic environment, the agent’s experience is divided into atomic episodes. Each episode consists of the agent perceiving and performing a single action. Crucially, the action taken in one episode does not affect decisions made in subsequent episodes. In a sequential environment, current decisions influence all future decisions, requiring long-term planning.

Static versus Dynamic

An environment is static if it does not change while the agent is deliberating. An environment is dynamic if it changes continuously while the agent processes information. If the environment itself does not change with time, but the agent’s performance score deteriorates as deliberation time passes, the environment is classified as semidynamic.

Discrete versus Continuous

An environment is discrete if it possesses a finite or countably infinite number of distinct states, percepts, and actions. An environment is continuous if states, percepts, or actions vary continuously over real numbers, such as vehicle positioning, velocity, and steering angles.

Known versus Unknown

This dimension refers to the agent designer’s state of knowledge regarding the environment’s operational laws. In a known environment, the outcomes or outcome probabilities for all actions are mathematically specified. In an unknown environment, the agent must learn how the environment functions through direct trial and interaction.

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Chess (with-out clock)	Fully	Multi	Deterministic	Sequential	Static	Discrete
Chess (with clock)	Fully	Multi	Deterministic	Sequential	Semidynamic	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Automated Taxi Driver	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical Diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Part-Sorting Robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Image Analysis	Fully	Single	Deterministic	Episodic	Semidynamic	Discrete

Table 2.3: Classification of sample task environments across major structural dimensions.

2.4 Agent Architectures and Program Design

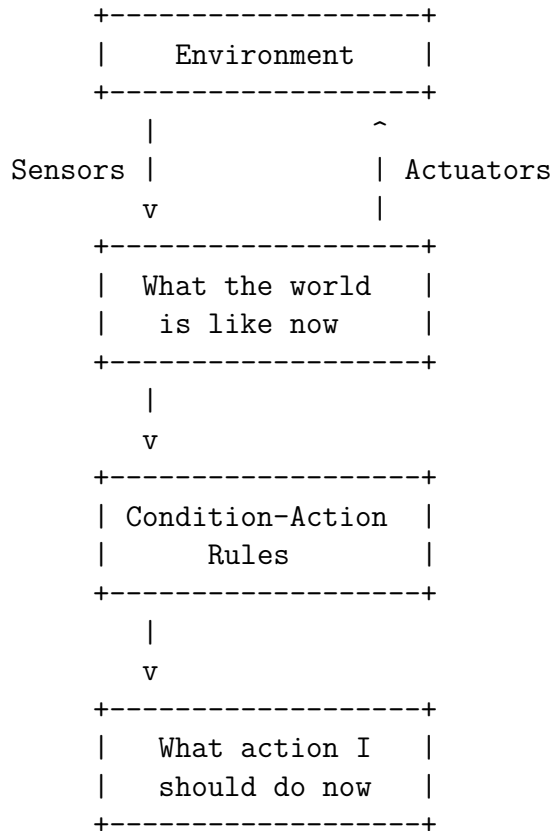
An agent architecture provides the physical computing system and sensor-actuator interface that executes the agent program. The relation between these components is expressed as:

$$\text{Agent} = \text{Architecture} + \text{Program} \quad (2.4)$$

The underlying structure of the agent program determines how percepts are transformed into concrete action sequences. Five basic agent architecture structures exist in artificial intelligence.

2.4.1 Simple Reflex Agents

Simple reflex agents select actions based exclusively on the current percept, ignoring the entire percept history. Their decision logic is governed by condition-action rules, often framed as simple if – then constructs.



The simple reflex agent function operates as follows:

```

function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition-action rules

  state <- INTERPRET-INPUT(percept)
  rule <- RULE-MATCH(state, rules)
  action <- rule.ACTION
  return action

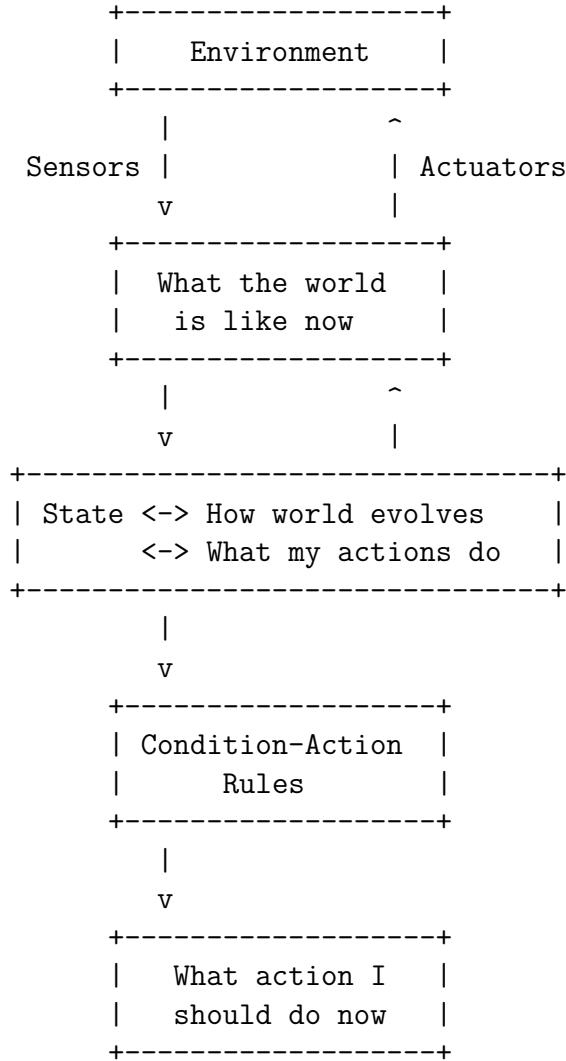
```

Simple reflex agents are limited because they can only operate successfully in environments that are fully observable. If deployed in partially observable domains, simple reflex agents frequently enter infinite loops.

2.4.2 Model-Based Reflex Agents

To handle partial observability, a model-based reflex agent maintains an internal state that tracks unobserved aspects of the current world. To maintain this state, the agent requires two types of knowledge encoded within its model:

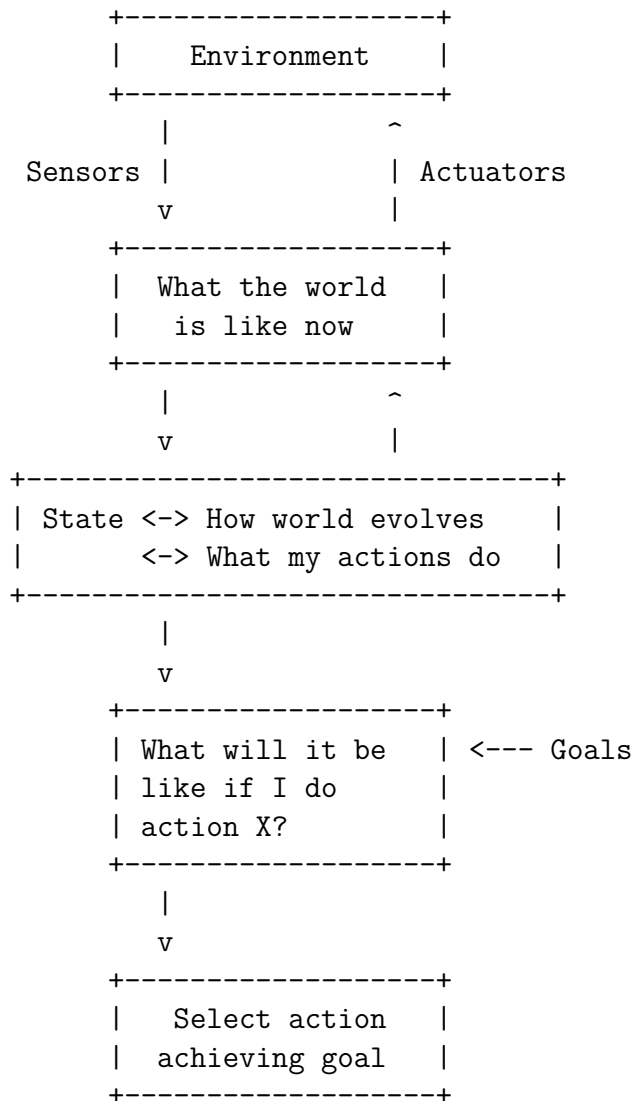
1. **Transition Model:** Knowledge about how the world evolves independently of the agent, and how the agent's actions affect the world.
2. **Sensor Model:** Knowledge about how the state of the world maps to sensory percepts.



The model-based agent updates its internal state representation over time using incoming percepts and its transition knowledge before selecting an action.

2.4.3 Goal-Based Agents

Knowing the current state of the environment is not always sufficient to decide what action to select. A goal-based agent combines internal state tracking with explicit goal specifications that describe desirable environment situations.

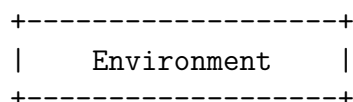


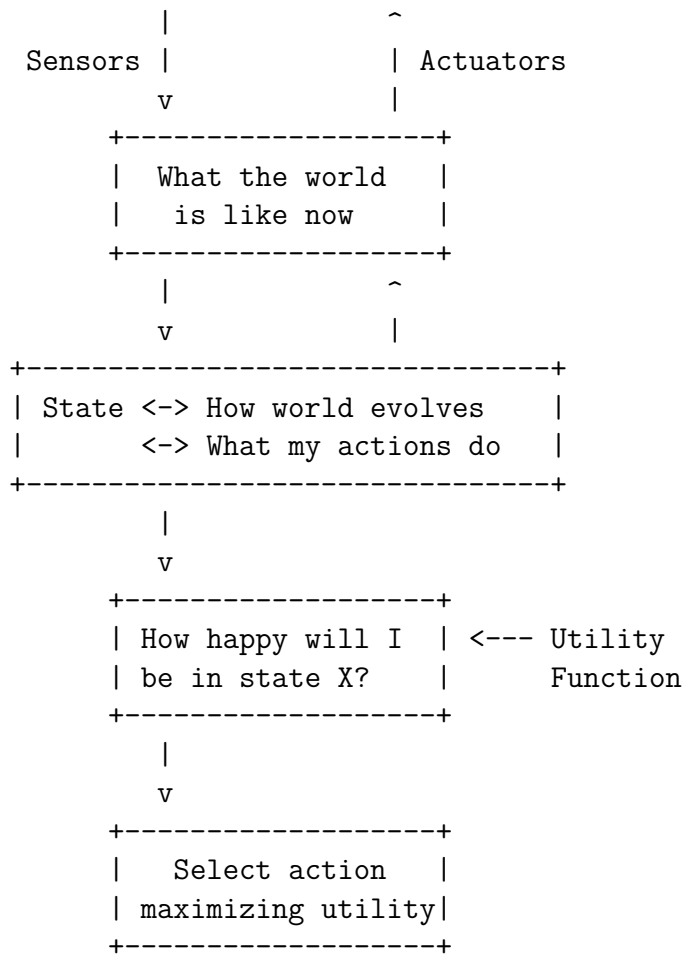
Goal-based architectures require deliberative mechanisms such as search algorithms and planning sequences. Unlike reflex agents, goal-based agents evaluate future outcomes before selecting an action, making them far more flexible when conditions change.

2.4.4 Utility-Based Agents

Goals provide a binary distinction between goal states and non-goal states. However, real-world decision-making requires evaluating trade-offs between competing outcomes, such as balance between speed, safety, and fuel efficiency in automated driving.

A utility-based agent utilizes a utility function that maps an environment state (or sequence of states) to a real number $U : S \rightarrow \mathbb{R}$. This numerical score measures how desirable a given state is for the agent.



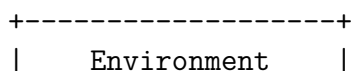


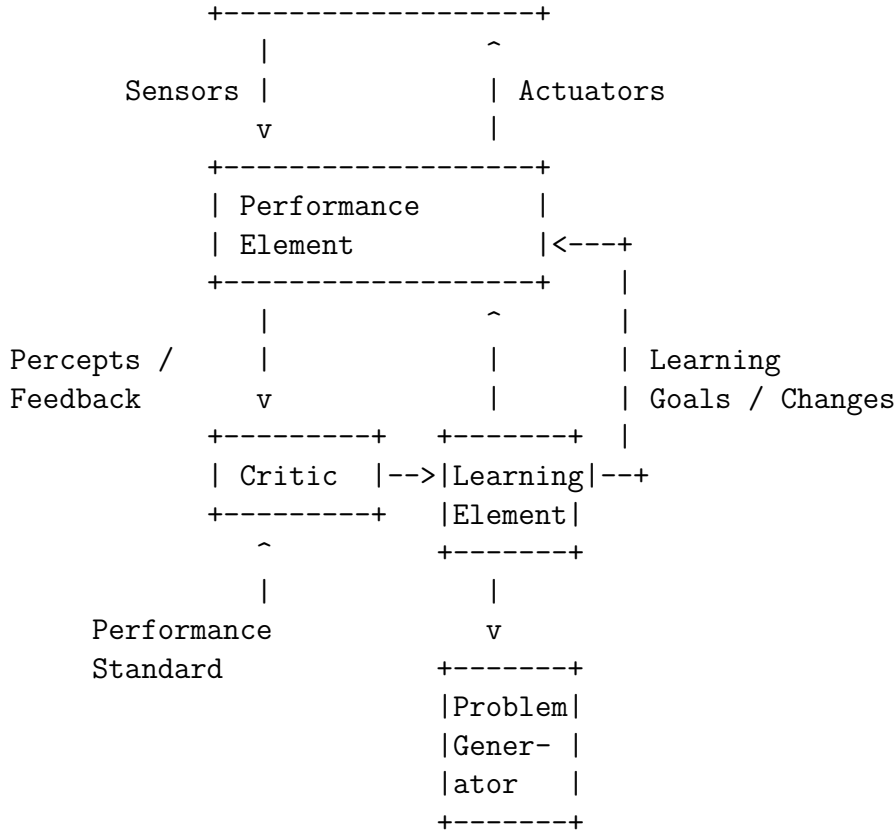
A utility-based agent selects actions that maximize expected utility, providing a principled basis for decision-making under uncertainty and resolving conflicts between mutually exclusive objectives.

2.4.5 Learning Agents

Learning agents adapt their internal decision logic automatically over time. A learning agent architecture is conceptually divided into four separate operational components:

- **Learning Element:** Responsible for making structural improvements to the agent program based on experience.
- **Critic:** Evaluates the agent's behavior against an external performance standard and provides evaluative feedback to the learning element.
- **Performance Element:** The component responsible for selecting external actions based on percepts; equivalent to the entire agent programs described in previous architectures.
- **Problem Generator:** Suggests exploratory actions that lead to new experiences rather than suboptimal actions, prioritizing discovery over short-term rewards.





2.4.6 Comparative Analysis of Agent Architectures

Table 2.4 summarizes the structural trade-offs between the five fundamental agent architectures.

Architecture	State Representation	Decision Mechanism	Best Suited Environment
Simple Reflex	None (Current percept only)	Condition-action rule lookup	Fully observable, static, simple domains
Model-Based Reflex	Internal state history	Rules evaluated over world model state	Partially observable domains with clear state logic
Goal-Based	Internal state history	Search and planning toward explicit goals	Complex domains requiring multi-step lookahead
Utility-Based	Internal state history	Utility optimization under probabilistic outcomes	Dynamic domains with conflicting trade-offs
Learning Agent	Dynamic adaptivity across components	Learning element guided by critic and problem generator	Unknown, non-stationary, complex environments

Table 2.4: Comparative breakdown of fundamental agent architectures.

2.5 Conclusion

The conceptual paradigm of intelligent agents provides a unified foundation for artificial intelligence. By formalizing agent interactions using the PEAS framework, defining rationality through expected performance maximization, and selecting appropriate agent architectures based on environmental dimensions, complex autonomous systems can be designed systematically. The agent framework established in this chapter serves as the theoretical foundation for Chapter 3, where specific algorithmic techniques for state space search and problem-solving are developed.

Tutorial Questions

1. Define an intelligent agent and explain the theoretical distinction between an agent function and an agent program. Why is implementing an agent function as a lookup table infeasible in practice?
2. Differentiate between rationality and omniscience using a concrete operational example. Identify the four factors that dictate rational behavior for an agent.
3. Specify the task environment for an automated medical diagnosis agent using the PEAS framework.
4. Classify the following task environments across all seven environmental dimensions:
 - (a) An automated warehouse forklift picking and moving boxes.
 - (b) An online fraud detection system monitoring credit card transactions.
 - (c) A software agent playing an online multi-player strategy game.
5. A simple reflex agent is deployed in a partially observable environment. Explain why this agent is vulnerable to getting stuck in infinite loops. Provide a small abstract environment scenario illustrating this failure mode.
6. Contrast goal-based agents with utility-based agents. Describe a specific scenario where a goal-based agent fails to yield an optimal decision, whereas a utility-based agent succeeds.
7. Detail the four main internal components of a learning agent architecture. Explain the role played by the problem generator component and why pure performance optimization can be detrimental to an agent's long-term adaptability.